

Experiment 'p_4_1_d_sum_odd_numbers' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_1_d_sum_odd_numbers

Natural language statement: null

Method signature: p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)

Ensures

- `sum == sum_odd(a, b)`

Functional Code Given

```
function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)`

The method should respect the following contract:

`ensures sum == sum_odd(a, b)`

The contract uses the following dafny code:

```
function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLFYhoDcVepiRNr1Wg23Gx1X62av

Model: gpt-5.5-2026-04-23

Created at: 1780652128

Finish reason: stop

Usage: 444 tokens in, and 3881 tokens out

```
{
  if a > b {
    sum := 0;
    assert sum_odd(a, b) == 0;
  } else {
    sum := 0;
    var i := a;
    while i <= b
      invariant a <= i <= b + 1
```

```

    invariant sum + sum_odd(i, b) == sum_odd(a, b)
    decreases b - i + 1
  {
    if i % 2 == 1 {
      assert sum_odd(i, b) == i + sum_odd(i + 1, b);
      sum := sum + i;
    } else {
      assert sum_odd(i, b) == sum_odd(i + 1, b);
    }
    assert sum + sum_odd(i + 1, b) == sum_odd(a, b);
    i := i + 1;
  }
  assert i == b + 1;
  assert sum_odd(i, b) == 0;
  assert sum == sum_odd(a, b);
}

```

Final Program

```

function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}

method p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)
  ensures sum == sum_odd(a, b)
{
  if a > b {
    sum := 0;
    assert sum_odd(a, b) == 0;
  } else {
    sum := 0;
    var i := a;
    while i <= b
      invariant a <= i <= b + 1
      invariant sum + sum_odd(i, b) == sum_odd(a, b)
      decreases b - i + 1
      {
        if i % 2 == 1 {
          assert sum_odd(i, b) == i + sum_odd(i + 1, b);
          sum := sum + i;
        } else {
          assert sum_odd(i, b) == sum_odd(i + 1, b);
        }
        assert sum + sum_odd(i + 1, b) == sum_odd(a, b);
        i := i + 1;
      }
    }
  }
}

```

```
}  
assert i == b + 1;  
assert sum_odd(i, b) == 0;  
assert sum == sum_odd(a, b);  
}  
}
```

Total Token Usage

Input tokens: 444

Output tokens: 3881

Reasoning tokens: 3581

Sum of 'total tokens': 4325

Experiment Timings

Overall Experiment started at 1780652127990, ended at 1780652233621, lasting 105631ms (105.63 seconds)

Iteration #1 started at 1780652127996, ended at 1780652233621, lasting 105625ms (105.63 seconds)